Gion en la Presentación:

- Trabajo practico final: Patrones de diseño: Estudio un patrón y lo muestro

Introducción: ¿Que es un patrón de diseño?

- Idea positiva (2):

- Dentro del mundo del desarrollo de software los programadores nos encontramos en una constante búsqueda de practicas que sean cada vez mas eficientes con el fin de conseguir un sistema robusto mantenible y escalable.
  Los desafíos contantes en la programación nos condujeron a concebir soluciones recurrentes dando así al concepto de **patrón de diseño**, está siendo una solución estandarizada, probada y reutilizables son una guía conceptual y/o plantillas que describen como solucionar o satisfacer necesidades de la manera más optima.

Patron de diseño: Decorador.

- Idea positiva (3):

- Hoy profundizaremos en el **Patrón de Diseño, Estructural,** conocido como **Decorator** el cual se enfoca en combinar clases y objetos para formar estructuras amplias y flexibles.
  Este patrón en particular nos permite añadir nuevas funcionalidades y responsabilidades a un objeto ya existente de forma dinámica frente a la herencia clásica.

- Presentar el problema que resuelve a partir de un ejemplo:

- Idea positiva (4): ¿Qué problema resuelve?

- Imaginemos que estamos programando un juego y estamos **abstrayendo** a las identidades de este y queremos que algunas varíen sus **comportamientos** según que lleven equipado en su inventario.
  Ejemplos:

- Idea positiva (5): visualización de las posibles entidades.

- Comunmente pensamos en usar la **herencia convencional** o el **Patron de Diseño** conocido como **Estratigy** pero si utilizamos cualquiera de estas 2 alternativas tendríamos un numero muy grande de combinaciones o una **Estrategia para cada combinación posibles.**

- Idea positiva (6): Problemas puntuales que soluciona:

- "breve aclaración de los problemas puntuales que resuelve"
  -Explosión de clases o Multigeometria de clases
  -Código duplicado
  -Inmutabilidad del comportamiento
  -Modificación de código existente
  -Alternativa a la herencia múltiple.

PROBLEMA	SOLUCIÓN CON DECORATOR
Explosión de Subclases (Rigidez de la herencia)	Evita crear una subclase para cada combinación. Permite combinar características dinámicamente.
Inflexibilidad en Tiempo de Ejecución	Añade o quita responsabilidades mientras la aplicación se ejecuta.
Violación del Principio de Responsabilidad Única	Separa las responsabilidades principales de las secundarias o de infraestructura.
Modificación de Código Existente	Extiende el comportamiento de clases sin alterar su código fuente original.
Alternativa a la Herencia Múltiple	Permite componer un objeto con múltiples capacidades sin los problemas de la herencia múltiple.

- Los Dolores que sufre este **Patron de Diseño.**
- Idea positiva (7)
- “Breve mención y aclaración de los problemas que tiene este patrón”.
  Señalar la obviedad de que este patrón de diseño su mejor ventaja es su mejor debilidad ya que si no se tiene conciencia al programar terminaras creando un código muy grande y engorroso ya que tienes la posibilidad de hacerlo crecer de manera exponencial

DOLOR DE CABEZA		CONSECUENCIA EN PRODUCCIÓN
Debugging Infernal		Múltiples capas ocultan el objeto base dificultan rastrear errores
Pérdida de Identidad		El cliente interactúa con el wrapper rompiendo comparaciones directas
Inicialización Compleja		Requiere envoltura manual recursiva repetitiva en código
Explosión de Clases		Cada regla de negocio nueva exige crear un archivo separado
Interfaces Frágiles		Si la interfaz raíz cambia obliga a refactorizar todos los decoradores

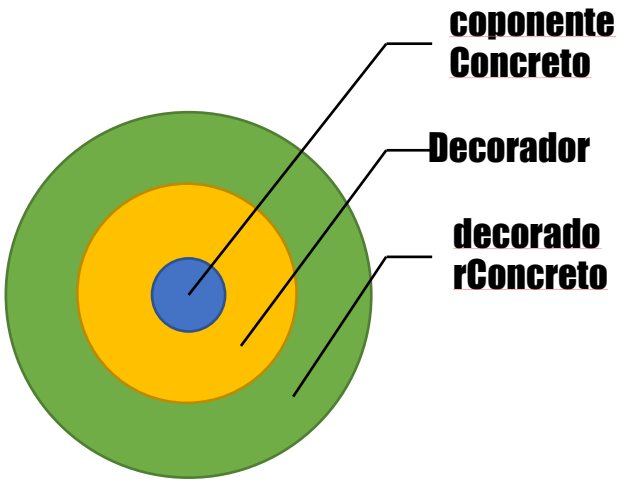
Diagrama U.M.L Genérico del patrón:

- Idea positiva (8):

➢ Ahora veremos como el patrón de diseño estructural funciona de forma formal a travez de un diagrama genérico en el en el formato u.m.l

- Idea positiva (9)

➢ Explicar de forma breve desde otra perspectiva el patrón ya que se lo conoce por otro nombre que al español se le conoce como envoltura dejando mas claro como es que este funciona desde el componenteConcreto envolviéndolo con el Decorador y volver a envolverlo nuevamente con un DecoradorConcreto disfrazando los comportamientos para poder conseguir nuevas combinaciones en un mismo código.



1° Primer Paso: Definición de la interfaz.

- Como implementarlo en Java. Idea positiva (10)

➢ Se establece un contrato con los **Métodos Básicos por** ejemplo **operador()**
Tanto el objeto real como los decoradores se comprometen a tener este método.

2° Segundo Paso: Creación del objeto Real

- Crear el objeto Real (componenteConcreto). Idea positiva (11)

➢ Este objeto implementa la lógica estándar y funciona de manera independiente
"También es el objeto que queremos envolver o añadir nuevas funcionales"

3° Tercer paso: Construcción del Envoltorio

- Crear la base para los decoradores (Decorador). Idea positiva (12)

➢ La idea princinpal es remplazar el componenteConcreto con un Decorador gracias a que ambos implementan la misma interfaz.
➢ Cuando ejecutes este método llamando a la operación al componente que está envolviendo, usando la función del componenteConcreto más una función propia del decoradorConcreto.

4° Cuarto Paso: Inyeccion de comportamiento.

- Crear los decoradores específicos(decoradoresConcretos)

➢ Estas clases se encargan de extender aun mas el comportamiento al sobre escribir, "al decorar" realizan las siguientes acciones.

-ejecutan código propio

-llaman al Ejecutado del objeto envuelto.

-Ejecutan más código propio.